

VISUAL BASIC .NET

Unit-3: Code Procedures, Event Handlers, Exception Handling, Data Validation & OOP

Comprehensive Study Notes

Himachal Pradesh University – BCA Semester IV

3.1 Code Procedures

What are Procedures?

Procedures are **blocks of code** that perform specific tasks. They help in:

- **Code Reusability** – Write once, use multiple times
- **Modularity** – Break complex problems into smaller parts
- **Maintainability** – Easy to update and debug
- **Readability** – Organized and structured code

Types of Procedures

Type	Description	Returns Value	Keyword
Sub Procedure	Performs actions	No	Sub
Function Procedure	Performs actions and returns a value	Yes	Function
Event Handler	Responds to events (special Sub)	No	Sub + Handles

3.1.1 Sub Procedures

A **Sub procedure** performs actions but does not return a value. It is declared with the `sub` keyword.

Syntax

```
[AccessModifier] Sub ProcedureName([ParameterList])  
    ' Code statements  
End Sub
```

Examples

```
' Simple Sub procedure  
Private Sub DisplayMessage()  
    MessageBox.Show("Hello, World!")  
End Sub  
  
' Sub with parameters  
Public Sub ShowGreeting(ByVal name As String)  
    MessageBox.Show("Hello, " & name)  
End Sub  
  
' Sub with multiple parameters  
Private Sub CalculateSum(ByVal a As Integer, ByVal b As Integer)  
    Dim result As Integer = a + b  
    MessageBox.Show("Sum: " & result.ToString())  
End Sub
```

3.1.2 Calling Sub Procedures

Sub procedures can be called with or without parentheses.

```
Private Sub CallingExamples()  
    ' Call without parentheses  
    DisplayMessage()  
  
    ' Call with parentheses  
    ShowGreeting("Alice")  
  
    ' Call with arguments  
    CalculateSum(10, 20)  
  
    ' Call using Call keyword (optional)  
    Call DisplayMessage()  
End Sub
```

Sub with ByRef Parameter

```
Private Sub DoubleValue(ByRef num As Integer)  
    num = num * 2  
End Sub  
  
Private Sub TestByRef()  
    Dim value As Integer = 5  
    DoubleValue(value)  
    ' value is now 10  
End Sub
```

3.1.3 Pass Arguments by Reference and by Value

ByVal (Pass by Value)

ByVal passes a *copy* of the variable. Changes inside the procedure do NOT affect the original. This is the **default** method.

```
Private Sub ChangeByVal(ByVal num As Integer)
    num = 100 ' Only changes local copy
End Sub

Private Sub TestByVal()
    Dim x As Integer = 10
    ChangeByVal(x)
    ' x is still 10
End Sub
```

ByRef (Pass by Reference)

ByRef passes a *reference (memory address)* to the variable. Changes affect the original variable.

```
Private Sub ChangeByRef(ByRef num As Integer)
    num = 100 ' Changes original variable
End Sub

Private Sub TestByRef2()
    Dim x As Integer = 10
    ChangeByRef(x)
    ' x is now 100
End Sub
```

Comparison Table

Aspect	ByVal	ByRef
Default	Yes	No
Original changed?	No	Yes
Memory	Creates copy	Uses original address

Safety	Safer (no side effects)	Can modify original
Use case	Simple values, strings (immutable)	Need to modify original, multiple return values

Guidelines

Scenario	Use	Reason
Simple values (Integer, Double)	ByVal	Prevents accidental modification
Strings	ByVal	Strings are immutable
Objects (Classes, Lists)	ByVal	Object reference passed, properties still modifiable
Need to modify original	ByRef	Must be explicit
Multiple return values	ByRef	Return values through parameters

Complete Example

```
Public Class Form1
    Private Sub Swap(ByRef a As Integer, ByRef b As Integer)
        Dim temp As Integer = a
        a = b
        b = temp
    End Sub

    Private Sub Calculate(ByVal x As Integer, ByRef y As Integer)
        x = x + 5
        y = y + 5
    End Sub

    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles
Button1.Click
        Dim p As Integer = 10
        Dim q As Integer = 20

        Swap(p, q)
        ' p=20, q=10

        Calculate(p, q)
        ' p=20, q=15 (p unchanged by ByVal, q changed by ByRef)
    End Sub
End Class
```

3.1.4 Function Procedures

A **Function procedure** performs actions AND returns a value. The return type must be specified.

Syntax

```
[AccessModifier] Function FunctionName([ParameterList]) As ReturnType
    ' Code statements
    Return returnValue
End Function
```

Examples

```
' Simple function
Private Function Add(ByVal a As Integer, ByVal b As Integer) As Integer
    Return a + b
End Function

' Function with string return
Public Function GetGreeting(ByVal name As String) As String
    Return "Hello, " & name & "!"
End Function

' Boolean function
Private Function IsEven(ByVal num As Integer) As Boolean
    Return num Mod 2 = 0
End Function

' Multiple parameters
Private Function CalculateArea(ByVal length As Double, ByVal width As Double) As Double
    Return length * width
End Function
```

Using Functions

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
    Dim sum As Integer = Add(15, 25)
    Dim msg As String = GetGreeting("Bob")
End Sub
```

```
    Dim area As Double = CalculateArea(5.5, 3.2)  
End Sub
```

3.1.5 Work with Events

Events are **actions** that occur in response to user interactions or system occurrences.

Events follow the **publisher-subscriber pattern**:

- **Publisher** (Control) raises the event
- **Subscriber** (Event Handler) responds to the event

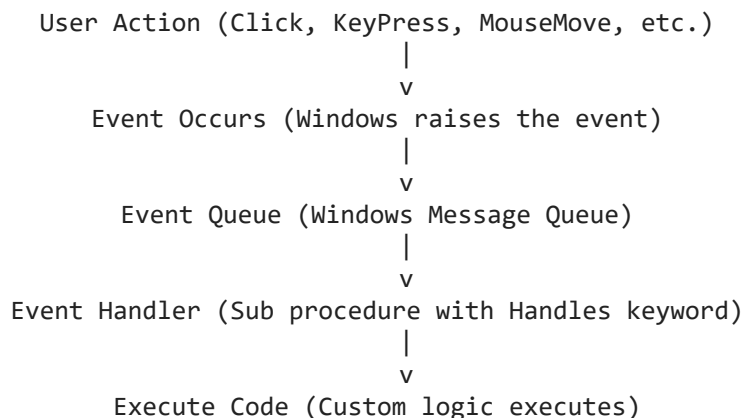
Event Handler Structure

```
' Standard Event Handler
Private Sub ControlName_EventName(sender As Object, e As EventArgs) _
    Handles ControlName.EventName
    ' Code to handle the event
End Sub
```

Common Events

Control	Default Event	Description
Button	Click	When button is clicked
TextBox	TextChanged	When text changes
Form	Load	When form first loads
ComboBox	SelectedIndexChanged	When selection changes
CheckBox	CheckedChanged	When checked state changes

Flowchart: Event Handler Lifecycle



3.1.6 Start an Event Handler for Any Event

Methods to Create Event Handlers

Method	Description
1. Double-click control	Creates default event handler in code editor
2. Properties Window – Events tab	Select control, click Events button, double-click event
3. Code Editor dropdowns	Left: select control, Right: select event
4. Manual coding	Write Sub with Handles keyword

Examples

```
' Method 1: Button Click (default)
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
    MessageBox.Show("Button clicked!")
End Sub

' Method 2: TextBox TextChanged
Private Sub TextBox1_TextChanged(sender As Object, e As EventArgs) Handles
    TextBox1.TextChanged
    MessageBox.Show("Text changed: " & TextBox1.Text)
End Sub

' Method 3: Form Load
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    MessageBox.Show("Form is loading...")
End Sub
```

3.1.7 Handle Multiple Events with One Handler

A single event handler can handle events from **multiple controls** by listing them after Handles .

```
' One handler for three buttons
Private Sub Button_Click(sender As Object, e As EventArgs) _
    Handles Button1.Click, Button2.Click, Button3.Click
    Dim btn As Button = CType(sender, Button)
    MessageBox.Show("You clicked: " & btn.Text)
End Sub

' One handler for multiple checkboxes
Private Sub CheckBox_CheckedChanged(sender As Object, e As EventArgs) _
    Handles CheckBox1.CheckedChanged, CheckBox2.CheckedChanged
    Dim chk As CheckBox = CType(sender, CheckBox)
    If chk.Checked Then
        MessageBox.Show(chk.Text & " is checked")
    End If
End Sub
```

Add and Remove Event Handlers Programmatically

```
' AddHandler
AddHandler Button1.Click, AddressOf Button_Click

' RemoveHandler
RemoveHandler Button1.Click, AddressOf Button_Click
```

3.1.8 Use Code Editor to Start an Event Handler

Visual Studio provides code editor dropdowns to quickly create event handlers:

1. Open Code Editor (double-click form or press F7)
2. Left dropdown (Classes): Select the control name
3. Right dropdown (Methods): Select the event name
4. Visual Studio automatically generates the event handler stub

3.1.9 The Function Procedure, Event Handlers, Message Box

MessageBox – Complete Reference

```
MessageBox.Show(text, title, buttons, icon, defaultButton)
```

MessageBox Buttons

Constant	Shows
MessageBoxButtons.OK	OK button
MessageBoxButtons.OKCancel	OK and Cancel
MessageBoxButtons.YesNo	Yes and No
MessageBoxButtons.YesNoCancel	Yes, No, Cancel
MessageBoxButtons.RetryCancel	Retry and Cancel
MessageBoxButtons.AbortRetryIgnore	Abort, Retry, Ignore

MessageBox Icons

Constant	Icon
MessageBoxIcon.Information	i (Information)
MessageBoxIcon.Warning	! (Warning)
MessageBoxIcon.Error	X (Error)
MessageBoxIcon.Question	? (Question)
MessageBoxIcon.Exclamation	! (Exclamation)

Examples

```
' Simple message
MessageBox.Show("Data saved successfully!")

' With title
MessageBox.Show("File not found.", "Error")

' With buttons and icon
```

```
Dim result As DialogResult = MessageBox.Show(  
    "Are you sure you want to exit?",  
    "Confirm Exit",  
    MessageBoxButtons.YesNo,  
    MessageBoxIcon.Question)  
  
If result = DialogResult.Yes Then  
    Application.Exit()  
End If
```

Complete Example – Function with Message Box

```
Public Class Form1  
    Private Function CalculateGrade(ByVal marks As Integer) As String  
        If marks >= 90 Then Return "A"  
        If marks >= 75 Then Return "B"  
        If marks >= 60 Then Return "C"  
        If marks >= 45 Then Return "D"  
        Return "F"  
    End Function  
  
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles  
Button1.Click  
        Dim marks As Integer = Convert.ToInt32(TextBox1.Text)  
        Dim grade As String = CalculateGrade(marks)  
        MessageBox.Show("Grade: " & grade, "Result", MessageBoxButtons.OK,  
MessageBoxIcon.Information)  
    End Sub  
End Class
```

3.1.10 Built-in Functions

String Functions

Function	Description	Example	Result
Len(str)	Returns length of string	Len("Hello")	5
Mid(str, start, len)	Extracts substring	Mid("Hello", 2, 3)	"ell"
Left(str, len)	Leftmost characters	Left("Hello", 2)	"He"
Right(str, len)	Rightmost characters	Right("Hello", 2)	"lo"
Replace(str, old, new)	Replaces substring	Replace("Hello", "l", "x")	"Hexxo"
UCase(str)	Converts to uppercase	UCase("Hello")	"HELLO"
LCase(str)	Converts to lowercase	LCase("Hello")	"hello"
Trim(str)	Removes leading/trailing spaces	Trim(" Hello ")	"Hello"
InStr(str1, str2)	Finds position of substring	InStr("Hello", "ll")	3
Split(str, delim)	Splits string into array	Split("a,b,c", ",")	{"a","b","c"}
Join(arr, delim)	Joins array into string	Join({"a","b"}, ",")	"a,b"

```

Private Sub StringFunctionExamples()
    Dim text As String = " Visual Basic .NET "
    Dim trimmed As String = Trim(text)
    Dim upper As String = UCase(trimmed)
    Dim length As Integer = Len(trimmed)

    Dim msg As String = "Hello World"
    Dim part As String = Mid(msg, 7, 5) ' "World"
    Dim leftPart As String = Left(msg, 5) ' "Hello"

    Dim csv As String = "apple,banana,orange"
    Dim fruits() As String = Split(csv, ",")
  
```

```

    Dim joined As String = Join(fruits, " | ")
End Sub

```

Math Functions

Function	Description	Example	Result
Abs(x)	Absolute value	Abs(-5)	5
Sqrt(x)	Square root	Sqrt(16)	4.0
Pow(x, y)	x raised to power y	Pow(2, 3)	8.0
Min(x, y)	Smaller of two values	Min(10, 20)	10
Max(x, y)	Larger of two values	Max(10, 20)	20
Round(x, digits)	Rounds to specified digits	Round(3.14159, 2)	3.14
Floor(x)	Rounds down	Floor(3.7)	3
Ceiling(x)	Rounds up	Ceiling(3.2)	4
Sin(x)	Sine of angle (radians)	Sin(0)	0.0
Cos(x)	Cosine of angle (radians)	Cos(0)	1.0
Tan(x)	Tangent of angle (radians)	Tan(0)	0.0

```

Private Sub MathFunctionExamples()
    Dim absolute As Integer = Abs(-10)           ' 10
    Dim root As Double = Sqrt(25)                ' 5.0
    Dim power As Double = Pow(3, 4)              ' 81.0
    Dim smallest As Integer = Min(45, 23)        ' 23
    Dim largest As Integer = Max(45, 23)         ' 45
    Dim rounded As Double = Round(7.8925, 2)     ' 7.89
    Dim floorVal As Double = Floor(9.99)         ' 9
    Dim ceilVal As Double = Ceiling(9.01)        ' 10
End Sub

```

Date/Time Functions

Function	Description	Example
Now	Current date and time	Now.ToString()

Today	Current date (time = midnight)	Today.ToString("d")
DateDiff(interval, d1, d2)	Difference between dates	DateDiff(DateInterval.Day, d1, d2)
DateAdd(interval, num, date)	Add interval to date	DateAdd(DateInterval.Day, 7, Today)
Format(date, formatStr)	Formats date/time	Format(Now, "dd/MM/yyyy")

```

Private Sub DateTimeExamples()
    Dim current As DateTime = Now
    Dim today As Date = Today
    Dim diff As Long = DateDiff(DateInterval.Day, #1/1/2026#, #7/11/2026#)
    Dim nextWeek As DateTime = DateAdd(DateInterval.Day, 7, Today)
    Dim formatted As String = Format(Now, "dddd, MMMM dd, yyyy")
End Sub

```

Conversion Functions

Function	Description	Example
CInt(value)	Converts to Integer	CInt("123")
CStr(value)	Converts to String	CStr(123)
Cdbl(value)	Converts to Double	Cdbl("45.67")
Val(value)	Returns numeric value from string	Val("123abc")

```

Private Sub ConversionExamples()
    Dim num As Integer = CInt("42")
    Dim str As String = CStr(42)
    Dim dbl As Double = Cdbl("3.14")
    Dim valNum As Double = Val("123 Main St") ' 123.0
End Sub

```

Format Functions

Function	Description	Example
Format(value, formatStr)	Custom format	Format(1234.5, "\$#,##0.00")

FormatCurrency(value)	Formats as currency	FormatCurrency(45.5)
FormatNumber(value)	Formats as number	FormatNumber(1234.567, 2)
FormatPercent(value)	Formats as percentage	FormatPercent(0.15)
FormatDateTime(date)	Formats date/time	FormatDateTime(Now)

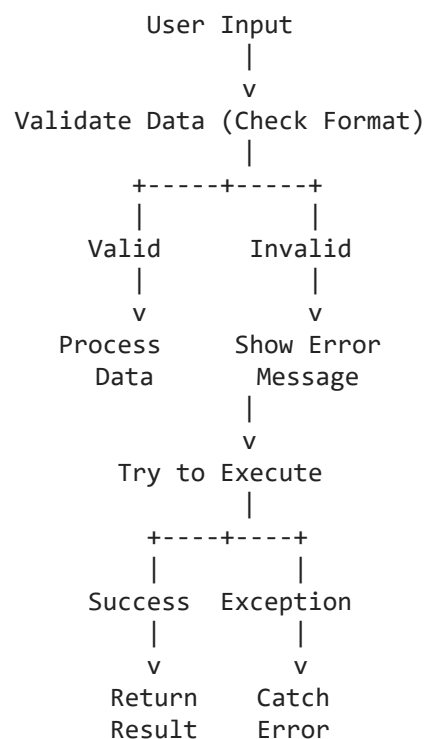
```
Private Sub FormatExamples()  
    Dim price As Double = 1234.5  
    Dim pct As Double = 0.15  
    Dim dt As DateTime = Now  
  
    Dim fCurrency As String = FormatCurrency(price)  
    Dim fNumber As String = FormatNumber(price, 2)  
    Dim fPercent As String = FormatPercent(pct)  
    Dim fDateTime As String = FormatDateTime(dt, DateFormat.LongDate)  
End Sub
```

3.2 Exception Handling & Data Validation

3.2.1 Introduction

Data validation is the process of checking user input to ensure it meets certain criteria before being processed. **Exception handling** is the process of responding to runtime errors in a controlled manner to prevent application crashes.

Validation and Exception Flow



3.2.2 Use the IsNumeric Function

The `IsNumeric` function checks if a string can be converted to a number.

`IsNumeric(expression)` ' Returns Boolean

```
Private Sub IsNumericExamples()  
    Dim input As String = TextBox1.Text  
  
    If IsNumeric(input) Then  
        Dim value As Double = CDb1(input)  
        MessageBox.Show("Valid number: " & value)  
    Else  
        MessageBox.Show("Please enter a valid number.", "Input Error",  
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)  
    End If  
End Sub
```

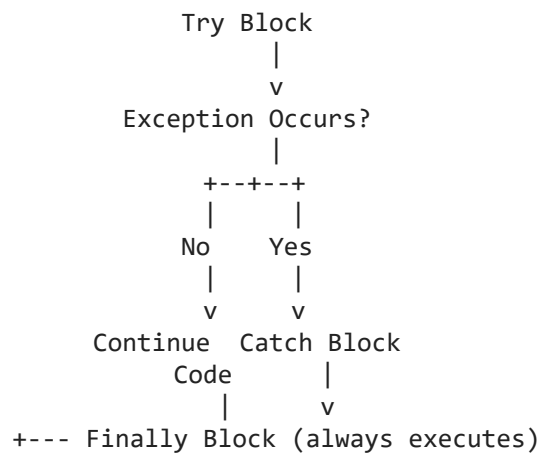
3.2.3 Display a Dialog Box for Error Messages

```
Public Class Form1
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles
Button1.Click
        If TextBox1.Text = "" Then
            MessageBox.Show("Please enter a value.", "Validation Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error)
            TextBox1.Focus()
        ElseIf Not IsNumeric(TextBox1.Text) Then
            MessageBox.Show("Please enter a numeric value.", "Invalid Input",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            TextBox1.SelectAll()
            TextBox1.Focus()
        Else
            MessageBox.Show("Valid input: " & TextBox1.Text, "Success",
                MessageBoxButtons.OK, MessageBoxIcon.Information)
        End If
    End Sub
End Class
```

3.2.4 Exception Handling – Structure

Try-Catch-Finally Block

```
Try
    ' Code that might cause an exception
Catch ex As ExceptionType
    ' Code to handle the exception
Finally
    ' Cleanup code (always executes)
End Try
```



3.2.5 Catch an Exception

Exception Properties

Property	Description	Example
Message	Error description	ex.Message
StackTrace	Call stack info	ex.StackTrace
Source	Application/object causing error	ex.Source
TargetSite	Method that threw exception	ex.TargetSite.Name
InnerException	Previous exception (nested)	ex.InnerException
HelpLink	Link to help file	ex.HelpLink
Data	Key-value pairs for additional info	ex.Data

Catching Examples

```
Private Sub DivisionExample()  
    Try  
        Dim num1 As Integer = CInt(TextBox1.Text)  
        Dim num2 As Integer = CInt(TextBox2.Text)  
        Dim result As Integer = num1 / num2  
        MessageBox.Show("Result: " & result)  
    Catch ex As DivideByZeroException  
        MessageBox.Show("Cannot divide by zero!", "Math Error",  
                        MessageBoxButtons.OK, MessageBoxIcon.Error)  
    Catch ex As FormatException  
        MessageBox.Show("Invalid number format!", "Input Error",  
                        MessageBoxButtons.OK, MessageBoxIcon.Warning)  
    Catch ex As Exception  
        MessageBox.Show("Unexpected error: " & ex.Message, "Error",  
                        MessageBoxButtons.OK, MessageBoxIcon.Error)  
    Finally  
        ' Always executes - cleanup code  
    End Try  
End Sub
```

3.2.6 Properties and Methods of an Exception – Class Hierarchy

```

        System.Object
        +-- System.Exception
        +-- System.SystemException
        |   +-- System.ArithmeticException
        |   |   +-- DivideByZeroException
        |   |   +-- OverflowException
        |   |   +-- NotFiniteNumberException
        |   +-- ArrayTypeMismatchException
        |   +-- IndexOutOfRangeException
        |   +-- InvalidCastException
        |   +-- NullReferenceException
        |   +-- OutOfMemoryException
        +-- System.ApplicationException
        |   +-- (Custom exceptions)
        +-- System.FormatException
    
```

```

Private Sub ExceptionPropertiesExample()
    Try
        Dim arr(4) As Integer
        arr(10) = 100 ' Will throw IndexOutOfRangeException
    Catch ex As Exception
        MessageBox.Show("Message: " & ex.Message & vbCrLf &
            "Source: " & ex.Source & vbCrLf &
            "Stack: " & ex.StackTrace,
            "Exception Details",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error)
    End Try
End Sub
    
```

Custom Exception Class

```

' Creating a custom exception
Public Class InvalidAgeException
    Inherits Exception

    Public Sub New()
        MyBase.New("Age must be between 0 and 150.")
    End Sub

    Public Sub New(ByVal message As String)
        MyBase.New(message)
    End Sub
    
```

```
End Class

' Using custom exception
Private Sub ValidateAge(ByVal age As Integer)
    If age < 0 OrElse age > 150 Then
        Throw New InvalidAgeException("Age " & age & " is out of range.")
    End If
End Sub
```

3.2.7 Throw an Exception

```
Public Class Form1
    Private Function DivideNumbers(ByVal a As Integer, ByVal b As Integer) As Integer
        If b = 0 Then
            Throw New ArgumentException("Divisor cannot be zero.")
        End If
        Return a / b
    End Function

    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click
        Try
            Dim result As Integer = DivideNumbers(10, 0)
        Catch ex As ArgumentException
            MessageBox.Show(ex.Message, "Error",
                            MessageBoxButtons.OK, MessageBoxIcon.Error)
        End Try
    End Sub
End Class
```

3.2.8 Application with Exception Handling

```
Public Class Form1
    Private Sub btnCalculate_Click(sender As Object, e As EventArgs) Handles
btnCalculate.Click
        Try
            Dim num1 As Double = Cdbl(txtNum1.Text)
            Dim num2 As Double = Cdbl(txtNum2.Text)
            Dim op As String = cboOperator.Text
            Dim result As Double = 0

            Select Case op
                Case "+" : result = num1 + num2
                Case "-" : result = num1 - num2
                Case "*" : result = num1 * num2
                Case "/" : result = num1 / num2
                Case Else : Throw New InvalidOperationException("Invalid operator")
            End Select

            lblResult.Text = "Result: " & result.ToString()
        Catch ex As FormatException
            MessageBox.Show("Enter valid numbers.", "Input Error",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Catch ex As DivideByZeroException
            MessageBox.Show("Cannot divide by zero.", "Math Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error)
        Catch ex As Exception
            MessageBox.Show(ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error)
        End Try
    End Sub
End Class
```

3.2.9 Validate Data – Techniques

Technique	Description
IsNumeric Function	Checks if value is numeric
TryParse Methods	Tries conversion, returns success/failure (no exceptions)
Regular Expressions	Pattern matching for complex validation (email, phone)
Validating Event	Occurs when control loses focus; can cancel focus change
MaskedTextBox	Prevents invalid input using a mask pattern

Validate a Single Entry

```
Public Class Form1
    Private Sub btnValidate_Click(sender As Object, e As EventArgs) Handles
        btnValidate.Click
            If txtName.Text.Trim() = "" Then
                ErrorProvider1.SetError(txtName, "Name is required")
                txtName.Focus()
            ElseIf Not IsNumeric(txtAge.Text) Then
                ErrorProvider1.SetError(txtAge, "Age must be numeric")
                txtAge.Focus()
            Else
                ErrorProvider1.Clear()
                MessageBox.Show("Validation passed!", "Success",
                                MessageBoxButtons.OK, MessageBoxIcon.Information)
            End If
        End Sub
    End Class
```

3.2.10 Use Generic Procedures to Validate an Entry

```
Public Class Form1
    Private Function IsValidName(ByVal name As String) As Boolean
        Return name.Trim().Length > 0 AndAlso Not IsNumeric(name)
    End Function

    Private Function IsValidAge(ByVal age As String) As Boolean
        If IsNumeric(age) Then
            Dim val As Integer = CInt(age)
            Return val >= 0 AndAlso val <= 150
        End If
        Return False
    End Function

    Private Function IsValidEmail(ByVal email As String) As Boolean
        Return email.Contains("@") AndAlso email.Contains(".")
    End Function

    Private Sub btnSubmit_Click(sender As Object, e As EventArgs) Handles
btnSubmit.Click
        If Not IsValidName(txtName.Text) Then
            MessageBox.Show("Enter a valid name.", "Validation Error")
        ElseIf Not IsValidAge(txtAge.Text) Then
            MessageBox.Show("Enter a valid age (0-150).", "Validation Error")
        ElseIf Not IsValidEmail(txtEmail.Text) Then
            MessageBox.Show("Enter a valid email.", "Validation Error")
        Else
            MessageBox.Show("All validations passed!", "Success")
        End If
    End Sub
End Class
```

3.2.11 Validate Multiple Entries

```
Public Class Form1
    Private Function ValidateAll() As Boolean
        Dim valid As Boolean = True
        ErrorProvider1.Clear()

        If txtName.Text.Trim() = "" Then
            ErrorProvider1.SetError(txtName, "Required")
            valid = False
        End If
        If Not IsNumeric(txtAge.Text) Then
            ErrorProvider1.SetError(txtAge, "Must be numeric")
            valid = False
        End If
        If txtEmail.Text.Trim() = "" OrElse Not txtEmail.Text.Contains("@") Then
            ErrorProvider1.SetError(txtEmail, "Valid email required")
            valid = False
        End If

        Return valid
    End Function

    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles
        btnSave.Click
            If ValidateAll() Then
                MessageBox.Show("Record saved successfully!", "Success")
            End If
        End Sub
    End Class
```

3.2.12 Application with Data Validation

```
Public Class MainForm
    Private Sub btnRegister_Click(sender As Object, e As EventArgs) Handles
btnRegister.Click
        If Not ValidateInputs() Then Exit Sub

        Try
            ' Process registration
            MessageBox.Show("Registration successful!", "Success",
                MessageBoxButtons.OK, MessageBoxIcon.Information)
        Catch ex As Exception
            MessageBox.Show("Error: " & ex.Message, "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error)
        End Try
    End Sub

    Private Function ValidateInputs() As Boolean
        If txtFullName.Text.Trim() = "" Then
            MessageBox.Show("Full name is required.", "Validation Error")
            txtFullName.Focus()
            Return False
        End If
        If Not IsNumeric(txtAge.Text) OrElse CInt(txtAge.Text) < 1 Then
            MessageBox.Show("Enter a valid age.", "Validation Error")
            txtAge.Focus()
            Return False
        End If
        If txtEmail.Text.Trim() = "" OrElse Not txtEmail.Text.Contains("@") Then
            MessageBox.Show("Enter a valid email.", "Validation Error")
            txtEmail.Focus()
            Return False
        End If
        Return True
    End Function
End Class
```

3.2.13 Difference Between Validating Event and MaskedTextBox

Feature	Validating Event	MaskedTextBox
Purpose	Custom validation logic	Pre-format input with mask
When it occurs	When control loses focus	During input (real-time)
Validation type	Programmatic	Pattern-based
Flexibility	High (any logic)	Limited (mask patterns)
Error handling	Custom error messages	Built-in validation
User experience	After entering data	During data entry
Complexity	More complex	Simple to use

Validating Event Example

```
Private Sub txtAge_Validating(sender As Object, e As
System.ComponentModel.CancelEventArgs) _
    Handles txtAge.Validating
    If Not IsNumeric(txtAge.Text) OrElse CInt(txtAge.Text) < 0 OrElse
CInt(txtAge.Text) > 150 Then
        MessageBox.Show("Enter a valid age (0-150).", "Invalid Input")
        e.Cancel = True ' Prevents focus from leaving
    End If
End Sub
```

MaskedTextBox Example

```
' Mask: 000-000-0000 (Phone number)
' Mask: 00/00/0000 (Date)
' Mask: >L0?L0L0? (License plate)
Private Sub Form1_Load(sender As Object, e As EventArgs) Handles MyBase.Load
    MaskedTextBox1.Mask = "000-000-0000" ' Phone
    MaskedTextBox2.Mask = "00/00/0000" ' Date
    MaskedTextBox3.Mask = ">L0?L0L0?L0" ' Custom
End Sub
```

3.3 Object-Oriented Programming (OOP) Concepts

3.3.1 Class and Module

A **Class** is a blueprint for creating objects. It defines properties, methods, and events. A **Module** is a container for shared members that can be accessed without creating an instance.

```
' Class definition
Public Class Person
    Public Property Name As String
    Public Property Age As Integer

    Public Sub New()
        Name = ""
        Age = 0
    End Sub

    Public Sub Display()
        MessageBox.Show("Name: " & Name & ", Age: " & Age.ToString())
    End Sub
End Class

' Module definition
Module MathHelper
    Public Function Square(ByVal x As Integer) As Integer
        Return x * x
    End Function
End Module

' Usage
Private Sub TestClass()
    Dim p As New Person()
    p.Name = "Alice"
    p.Age = 25
    p.Display()

    Dim sq As Integer = MathHelper.Square(5) ' No instance needed
End Sub
```

3.3.2 Namespace

A **Namespace** organizes classes and prevents naming conflicts.

```

Namespace College
    Public Class Student
        Public Property RollNo As Integer
        Public Property Name As String
    End Class

    Public Class Course
        Public Property Code As String
        Public Property Title As String
    End Class
End Namespace

' Using the namespace
Imports College

Private Sub TestNamespace()
    Dim s As New Student()
    s.RollNo = 101
    s.Name = "John"
End Sub

```

3.3.3 Access Modifiers

Modifier	Description	Scope
Public	No restrictions	Anywhere
Private	Accessible only within declaring block	Containing type
Protected	Accessible within class and derived classes	Class + derived
Friend	Accessible within same assembly	Same assembly
Protected Friend	Protected OR Friend (wider of both)	Same assembly + derived

```

Public Class Example
    Public Property PublicProp As String      ' Anywhere
    Private Property PrivateProp As String    ' Same class only
    Protected Property ProtectedProp As String ' Class + derived classes
    Friend Property FriendProp As String       ' Same assembly
    Protected Friend Property Both As String   ' Friend + Protected
End Class

```

3.3.4 Properties – Get/Set Accessors

```
Public Class Employee
    Private _name As String
    Private _salary As Double

    ' Auto-implemented property
    Public Property ID As Integer

    ' Full property with validation
    Public Property Name As String
        Get
            Return _name
        End Get
        Set(value As String)
            If value IsNot Nothing AndAlso value.Trim().Length > 0 Then
                _name = value
            Else
                Throw New ArgumentException("Name cannot be empty.")
            End If
        End Set
    End Property

    ' Read-only property
    Public ReadOnly Property Salary As Double
        Get
            Return _salary
        End Get
    End Property
End Class
```

3.3.5 Constructors (New)

```
Public Class Product
    Public Property Code As String
    Public Property Price As Decimal

    ' Default constructor
    Public Sub New()
        Code = ""
        Price = 0
    End Sub
End Class
```

```
' Parameterized constructor
Public Sub New(ByVal code As String, ByVal price As Decimal)
    Me.Code = code
    Me.Price = price
End Sub

' Copy constructor
Public Sub New(ByVal other As Product)
    Me.Code = other.Code
    Me.Price = other.Price
End Sub
End Class

' Constructor usage
Private Sub TestConstructors()
    Dim p1 As New Product()
    Dim p2 As New Product("P001", 25.99D)
    Dim p3 As New Product(p2)
End Sub
```

3.3.6 Inheritance

Inheritance allows a class (derived) to inherit members from another class (base). VB.NET uses the `Inherits` keyword.

Keyword	Description
Inherits	Derives a class from a base class
MyBase	Refers to the base class members
MustInherit	Abstract class (cannot be instantiated)
NotInheritable	Sealed class (cannot be inherited)
Overridable	Method can be overridden in derived class
Overrides	Derived class method overrides base method
Overloads	Method with same name but different parameters
Shadows	Hides a base class member

```
' Base class
Public MustInherit Class Animal
    Public Property Name As String
```

```
Public Sub New(ByVal name As String)
    Me.Name = name
End Sub

Public Overridable Sub Speak()
    MessageBox.Show("Animal speaks.")
End Sub

Public MustOverride Function GetInfo() As String
End Class

' Derived class
Public NotInheritable Class Dog
    Inherits Animal

    Public Sub New(ByVal name As String)
        MyBase.New(name)
    End Sub

    Public Overrides Sub Speak()
        MessageBox.Show(Name & " says: Woof!")
    End Sub

    Public Overrides Function GetInfo() As String
        Return "Dog: " & Name
    End Function
End Class

' Another derived class
Public Class Cat
    Inherits Animal

    Public Sub New(ByVal name As String)
        MyBase.New(name)
    End Sub

    Public Overrides Sub Speak()
        MessageBox.Show(Name & " says: Meow!")
    End Sub

    Public Overrides Function GetInfo() As String
        Return "Cat: " & Name
    End Function
End Class
```

```
' Usage
Private Sub TestInheritance()
    Dim animals As New List(Of Animal)()
    animals.Add(New Dog("Buddy"))
    animals.Add(New Cat("Whiskers"))

    For Each a As Animal In animals
        a.Speak()
    Next
End Sub
```

3.3.7 Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common base class. The correct method is called based on the actual object type at runtime.

```
Public MustInherit Class Shape
    Public MustOverride Function Area() As Double
    Public Overridable Sub Display()
        MessageBox.Show("Shape area: " & Area())
    End Sub
End Class

Public Class Circle
    Inherits Shape
    Public Property Radius As Double

    Public Sub New(ByVal r As Double)
        Radius = r
    End Sub

    Public Overrides Function Area() As Double
        Return Math.PI * Radius * Radius
    End Function
End Class

Public Class Rectangle
    Inherits Shape
    Public Property Width As Double
    Public Property Height As Double

    Public Sub New(ByVal w As Double, ByVal h As Double)
        Width = w
    End Sub
End Class
```

```

        Height = h
    End Sub

    Public Overrides Function Area() As Double
        Return Width * Height
    End Function
End Class

' Polymorphism in action
Private Sub TestPolymorphism()
    Dim shapes As New List(Of Shape)()
    shapes.Add(New Circle(5))
    shapes.Add(New Rectangle(4, 6))

    For Each s As Shape In shapes
        s.Display() ' Calls appropriate Display for each type
    Next
End Sub

```

3.3.8 Interfaces

An **Interface** defines a contract that implementing classes must follow. All interface members are implicitly `Public` and `MustOverride`.

```

' Interface declaration
Public Interface IComparable
    Function CompareTo(ByVal other As Object) As Integer
End Interface

Public Interface IPrintable
    Sub Print()
    Function GetPrintString() As String
End Interface

' Implementing interfaces
Public Class Book
    Implements IComparable, IPrintable

    Public Property Title As String
    Public Property Pages As Integer

    Public Function CompareTo(ByVal other As Object) As Integer _
        Implements IComparable.CompareTo
        Dim b As Book = CType(other, Book)

```

```
        Return Me.Title.CompareTo(b.Title)
    End Function

    Public Sub Print() Implements IPrintable.Print
        MessageBox.Show(GetPrintString())
    End Sub

    Public Function GetPrintString() As String _
        Implements IPrintable.GetPrintString
        Return Title & " (" & Pages & " pages)"
    End Function
End Class
```

Questions – Unit 3

Section A: Procedures

- Q:** What are procedures? Explain the different types of procedures in VB.NET.
-
- Q:** Write the syntax for declaring a Sub procedure. Give an example with parameters.
-
- Q:** Explain the difference between ByVal and ByRef parameter passing with examples.
-
- Q:** What is a Function procedure? How is it different from a Sub procedure?
-
- Q:** Write a VB.NET program to calculate the factorial of a number using a Function procedure.
-
- Q:** Explain event handlers with the Handles keyword. Write an example.
-
- Q:** How can you handle multiple events with a single event handler?
-
- Q:** Write a note on MessageBox.Show() with all its parameters.
-
- Q:** Explain the use of InputBox function with a suitable example.
-

Section B: Built-in Functions

- Q:** Explain any five String functions in VB.NET with examples.
-
- Q:** Write a program to demonstrate the use of Len, Mid, Left, and Right functions.
-
- Q:** Explain the Math functions: Abs, Sqrt, Pow, Min, Max, Round, Floor, Ceiling.
-
- Q:** What are Date/Time functions? Explain Now, Today, DateDiff, and DateAdd.
-
- Q:** Explain Format, FormatCurrency, FormatNumber, FormatPercent, and FormatDateTime.
-
- Q:** Write a program to convert a string to Integer, Double, and String using conversion functions.
-
- Q:** How does the Split function work? Give an example.
-

Section C: Exception Handling

Q: What is exception handling? Explain the Try-Catch-Finally structure.

Q: Differentiate between On Error GoTo and structured exception handling.

Q: Explain the common exception classes in .NET with their hierarchy.

Q: How do you throw an exception manually using the Throw keyword? Give an example.

Q: Write a program to handle DivideByZeroException and FormatException.

Q: What are the properties of the Exception class? Explain with an example.

Q: Create and use a custom exception class in VB.NET.

Section D: Data Validation

Q: What is data validation? Explain different validation techniques in VB.NET.

Q: How does the IsNumeric function work? Give an example.

Q: Explain the Validating event with an example.

Q: What is a MaskedTextBox? How is it different from the Validating event?

Q: Write a program to validate name, age, and email fields using generic procedures.

Q: How can you use ErrorProvider to display validation errors?

Section E: OOP Concepts

Q: What is a class? How is a class different from a module?

Q: Explain access modifiers in VB.NET: Public, Private, Protected, Friend, Protected Friend.

Q: What are properties? Explain Get and Set accessors with examples.

Q: What is a constructor? Explain parameterized constructors with examples.

Q: Explain inheritance in VB.NET. Write a program demonstrating base and derived classes.

Q: What is the difference between Overridable, Overrides, Overloads, and Shadows?

Q: Explain polymorphism with a suitable example.

Q: What is an interface? How do you implement an interface in VB.NET?

Q: Differentiate between MustInherit and NotInheritable classes.

Q: Write a program demonstrating the use of MyBase keyword.
